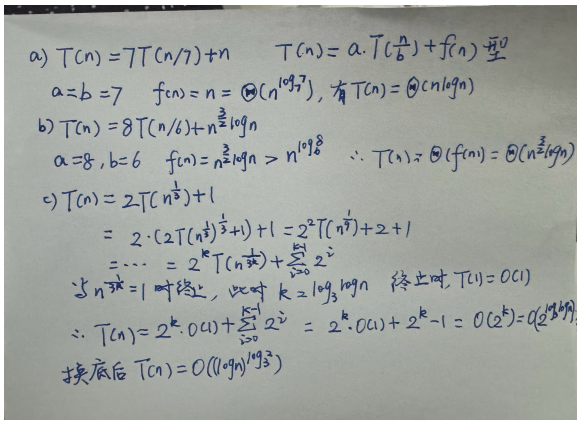
1. 求解下面的递归式，并针对每个递推式给出一个 Θ 界限。（20 分）
   a) $T(n)=7T(n/7)+n$ （6 分）
   b) $T(n)=8T(n/6)+n^{3/2}\log n$ （7 分）
   c) $T(n)=2T(n^{1/3})+1$ （7 分）



2. 给出一个求解背包问题的算法。并说明其正确性。（20 分）
   输入：给定 n 个物品，物品价值分别为 P_1，P_2，…，P_n，物品重量分别 W_1，W_2, …, W_n，背包容量为 M。每种物品可部分装入到背包中。
   输出：X_1，X_2，…，X_n，$0 \le X_i \le 1$，使得 $\sum_{1 \le i \le n} P_i X_i$ 最大，并且满足 $\sum_{1 \le i \le n} W_i X_i \le M$。

输入：

- n：物品个数。
- $P[i]$：第 i 个物品的价值。
- $W[i]$：第 i 个物品的重量。
- M：背包容量。

输出：

- $X[i]$：每种物品的装入比例，满足 $0 \le X[i] \le 1$。

算法步骤：

1. **计算单位重量价值**：
   - 计算每种物品的单位重量价值 $V[i] = P[i]/W[i]$。
2. **按单位重量价值降序排序**：
   - 将物品按 $V[i]$ 从大到小排序。
3. **依次装入背包**：
   - 初始化剩余背包容量 $remaining = M$。
   - 对于排序后的每个物品：
     - 如果物品重量 $W[i] \le remaining$，将整个物品装入背包：$X[i] = 1$，并更新剩余容量：$remaining -= W[i]$。
     - 如果物品重量 $W[i] > remaining$，只能部分装入：$X[i] = remaining/W[i]$，然后结束。
4. **输出结果**：
   - 返回 $X[i]$ 的比例数组。

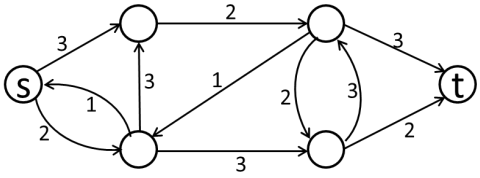
```python
def fractional_knapsack(P, W, M):
    n = len(P)
    # 计算单位重量价值
    value_per_weight = [(P[i] / W[i], i) for i in range(n)]
    # 按单位重量价值降序排序
    value_per_weight.sort(reverse=True, key=lambda x: x[0])

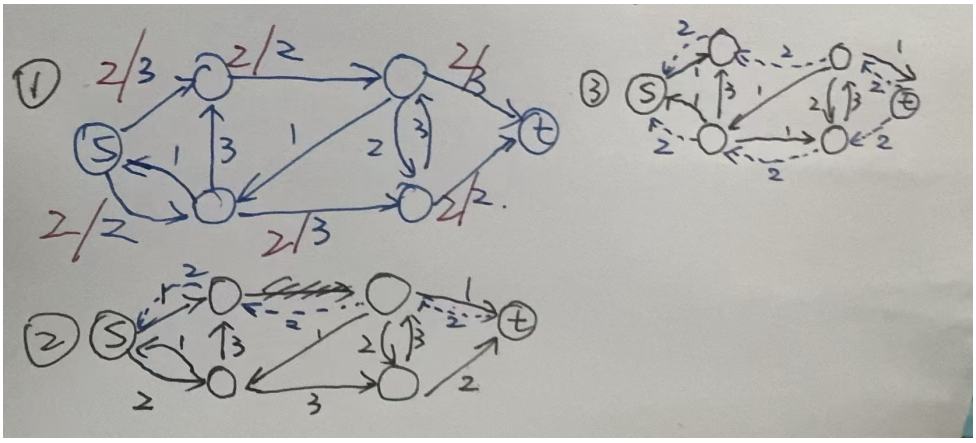
    X = [0] * n  # 初始化结果数组
    remaining = M  # 背包剩余容量

    for vpw, i in value_per_weight:
        if W[i] <= remaining:
            X[i] = 1  # 装入整个物品
            remaining -= W[i]
        else:
            X[i] = remaining / W[i]  # 部分装入
            break

    return X
```

3. 下图是一个流网络，请说明 s 到 t 的最大流值是多少？并标示出各边的流量。（15 分）





4. 设 $X[0:n-1]$和 $Y[0:n-1]$为两个数组，每个数组中含有 n 个已排好序的数。试设计一个 $O(\log n)$时间的分治算法，找出 X 和 Y 的 $2n$ 个数的中位数，并证明算法的时间复杂性为 $O(\log n)$。（15 分）

1. 中位数的定义是：在一个有序数组中，位于中间的数。如果总数为偶数，则中位数是中间两个数的平均值。
2. 给定两个已排序数组 X 和 Y，可以通过比较它们的中位数来缩小问题规模：
   - 比较 X 和 Y 的中位数 m_X 和 m_Y。
   - 如果 $m_X < m_Y$，那么最终的中位数一定在 $X[m_X : n-1]$ 和 $Y[0 : m_Y]$ 中；
   - 如果 $m_X > m_Y$，则最终中位数一定在 $X[0 : m_X]$ 和 $Y[m_Y : n-1]$ 中。
3. 不断递归地缩小数组的范围，直到两个数组的长度为 1，此时可以直接计算出中位数。

```
def find_median_sorted_arrays(X, Y, n):
    # 假设 X 和 Y 的长度均为 n
    if n == 1:
        # 递归终止条件，两个数组只剩一个元素
        return (max(X[0], Y[0]) + min(X[1], Y[1])) / 2

    # 找到 X 和 Y 的中位数
    mid_X = n // 2
    mid_Y = n // 2

    # 中位数的值
    m_X = X[mid_X]
    m_Y = Y[mid_Y]

    # 缩小问题规模
    if m_X == m_Y:
        return m_X  # 如果两个中位数相等，直接返回
    elif m_X < m_Y:
        # 舍弃 X 的左半部分 和 Y 的右半部分
        return find_median_sorted_arrays(X[mid_X:], Y[:mid_Y + 1], n - mid_X)
    else:
        # 舍弃 X 的右半部分 和 Y 的左半部分
        return find_median_sorted_arrays(X[:mid_X + 1], Y[mid_Y:], n - mid_Y)
```

伪代码

5. 给出 N 个 1-9 的数字 $(v_1,v_2,\ldots,v_N)$，不改变它们的相对位置，在中间加入 K 个乘号和 N-K-1 个加号，（括号随便加）使最终结果尽量大。因为乘号和加号一共就是 N-1 个了，所以恰好每两个相邻数字之间都有一个符号。并说明其具有优化子结构性质及子问题重叠性质。（20 分）

例如： N=5, K=2，5 个数字分别为 1、2、3、4、5，可以加成：

1*2*(3+4+5)=24

1*(2+3)*(4+5)=45

(1*2+3)*(4+5)=45

1. 问题分解

- **状态定义**：
设 $dp[i][k]$ 表示从数字 v_1 到 v_i 的子数组，插入 k 个乘号后所能得到的最大值。

- **转移方程**：
在第 j 个位置之前插入最后一个乘号时，分成两部分：

  - **左部分**：从 v_1 到 v_{j-1}，使用 $k-1$ 个乘号，结果为 $dp[j-1][k-1]$。
  - **右部分**：从 v_j 到 v_i，使用加号累加结果，结果为 $\text{sum}(v_j, v_j+1, , v_i)$。

因此，状态转移方程为：

$$dp[i][k] = \max_{j=1}^{i} \left(dp[j-1][k-1] \times \text{sum}(v_j, , v_i) \right)$$

其中，$\text{sum}(v_j, , v_i)$ 是从 v_j 到 v_i 的累加和。

- **初始状态**：

  - $dp[i][0]$：如果没有乘号，所有数字直接相加：

$$dp[i][0] = \text{sum}(v_1, , v_i)$$

- **目标**：
最终答案为 $dp[N][K]$，即使用 K 个乘号，覆盖整个数字数组的最大值。

- **优化子结构：**

由于在第 j 个位置分割的结果取决于前面的最优解 dp[j-1][k-1]，问题可以被分解为规模更小的子问题。整体问题的最优解可以由这些子问题的最优解组合得到。

- **子问题重叠：**

不同的分割方式会重复计算相同范围内的累加和（如 \text{sum}(v_j, ..., v_i)）。因此，通过动态规划表存储中间结果，避免重复计算。

6. 给定 n 个任务以及 2 个机器，每个任务所需的处理时间为 $p_1, p_2, \ldots, p_n$。一个任务可由任一台机器执行，但不能拆开分配给多台机器执行。一台机器在一时间只能处理一个任务。有一算法，它将这 n 个任务依次分配给这 2 个机器。算法在分配一任务时，总是将任务分配给到目前为止分配了最少工作时间的机器。请给出该算法的近似比，并证明。（10 分）

算法描述

贪心算法将任务依次分配到当前工作时间最少的机器上：

1. 初始化 $T_1 = 0, T_2 = 0$。
2. 对每个任务 p_i：
   - 如果 $T_1 \le T_2$，将任务分配到 M_1，并更新 $T_1 = T_1 + p_i$。
   - 否则，将任务分配到 M_2，并更新 $T_2 = T_2 + p_i$。

1. 最优解的定义

令 OPT 为最优分配下两台机器的最小最大完成时间，即：

$$OPT = \min \max(T_1, T_2)$$

显然，$OPT \ge \frac{\sum_{i=1}^{n} p_i}{2}$，因为两台机器的总工作时间至少要完成所有任务。

2. 贪心算法的性能

假设贪心算法的输出为 $T_{greedy} = \max(T_1, T_2)$。我们证明：

$$T_{greedy} \le \frac{3}{2} \cdot OPT$$

即近似比为 $\frac{3}{2}$。

东 南 大 学 考 试 卷（2015 春）